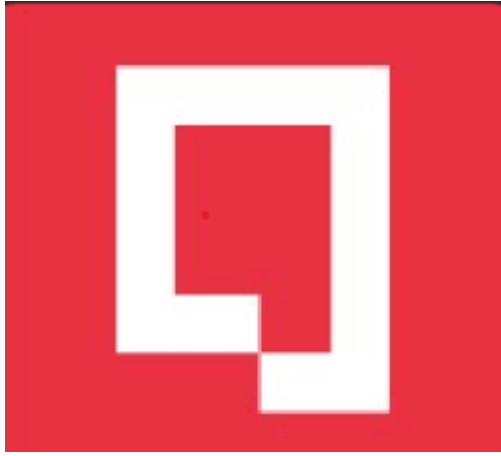




## 01 PAO25-25 - PYTHON, Data Types



Luis Pilaguano

### El Arreglo de Numpy

El artículo explica que en Python los arreglos de NumPy son la forma estándar de manejar datos numéricos. Su importancia está en que permiten realizar cálculos de manera rápida y eficiente dentro de un lenguaje de alto nivel como Python. Para lograr este rendimiento, se apoyan en tres ideas principales:

Para lograr este rendimiento, se apoyan en tres ideas principales:

1. Vectorización de cálculos: hacer operaciones sobre conjuntos de datos completos en lugar de usar bucles.
2. Evitar copias innecesarias de memoria: trabajar directamente sobre los bloques de memoria ya asignados.
3. Reducir la cantidad de operaciones: optimizar la forma en que se ejecutan los cálculos.

Un arreglo de NumPy no es más que una colección uniforme y multidimensional de elementos, que pueden ser números, booleanos, fechas, etc. Se caracteriza por dos cosas: el tipo de datos que contiene y su forma (shape). A diferencia de una simple

matriz, puede tener cualquier cantidad de dimensiones. El artículo también menciona que NumPy se originó en los años 90, gracias a un grupo de voluntarios que buscaban una estructura de datos adecuada para el cálculo científico. Hoy en día, NumPy es utilizado en universidades, laboratorios e industria, con aplicaciones que van desde videojuegos hasta la exploración espacial.

En cuanto al uso, se destacan funciones básicas como la indexación y el slicing para acceder a partes específicas de un arreglo. Por ejemplo, obtener filas, columnas o seleccionar cada cierto número de elementos, siempre recordando que Python empieza a contar desde cero

## Uso Básico

En todos los ejemplos de código de este artículo, asumimos que NumPy se importa de la siguiente manera:

```
In [1]: import numpy as np
```

## La estructura de un arreglo de NumPy: una vista sobre la memoria

En NumPy, los strides indican cuántos bytes se deben recorrer en memoria para llegar al siguiente dato, ya sea en filas o columnas. Por ejemplo, en un arreglo de enteros de 8 bytes cada uno, si tenemos una matriz 3x3, el stride de las filas será 24 bytes ( $3 \times 8$ ) y el de las columnas 8 bytes.

Esto hace posible que un mismo bloque de memoria se pueda ver de distintas formas sin necesidad de copiarlo.

Un ejemplo es cuando usamos slicing:

```
In [3]: x = np.arange(9).reshape((3, 3))
y = x[:, ::2, ::2]
```

Aquí y no crea un nuevo arreglo, sino que es una vista de x que muestra cada segundo elemento. Si cambiamos un valor en y, también se modifica en x, porque ambos comparten la misma memoria.

Además, los strides permiten operaciones como transponer (x.T) o cambiar la forma (x.reshape) sin mover datos de un lugar a otro. Por ejemplo, una matriz 3x3 puede convertirse en un arreglo de 1x9 en un instante, ya que lo único que cambia es la manera en que se interpreta la memoria.

En conclusión, los strides hacen que NumPy sea tan eficiente: permiten reorganizar y manipular los datos directamente en memoria sin necesidad de duplicarlos, lo que ahorra tiempo y espacio.

## Operaciones numéricas sobre matrices: vectorización

En cualquier lenguaje de scripting, el uso imprudente de bucles for puede llevar a un bajo rendimiento, particularmente en el caso donde se aplica un cálculo simple a cada elemento de un conjunto de datos grande.

Agrupar estas operaciones elemento por elemento, en un proceso conocido como vectorización, permite que NumPy realice dichos cálculos mucho más rápido.

## Ejemplos de vectorización y difusiónEvaluación de funciones

En Python, aplicar operaciones a cada elemento de un arreglo usando for-loops puede ser muy lento cuando se trabaja con grandes conjuntos de datos. NumPy resuelve esto con vectorización, que permite aplicar funciones a todo el arreglo de manera rápida y eficiente, ya que las operaciones están implementadas en C.

Además, NumPy permite realizar operaciones in-place, es decir, modificar los datos directamente en memoria sin crear arreglos temporales. Esto mejora aún más la velocidad y reduce el uso de memoria.

Por otro lado, herramientas como Cython, Theano o numexpr pueden optimizar todavía más el rendimiento en casos donde el código no se puede vectorizar fácilmente.

## Ejemplo práctico

Supongamos que queremos calcular la función:

$$f(x) = x^2 - 3x + 4$$

Método tradicional (for-loop)

```
In [12]: import numpy as np

# Función simple
def f(x):
    return x**2 - 3*x + 4

# Arreglo de 10 elementos para ver el resultado rápido
x = np.arange(10) # [0,1,2,...,9]

# Método tradicional con for-loop
y_loop = [f(i) for i in x]
print("For-loop:", y_loop)

# Vectorizado con NumPy
x_array = np.array(x)
y_vector = f(x_array)
print("Vectorizado:", y_vector)

# Vectorizado in-place
def g(x):
    fx = x**2          # crea un arreglo
    fx -= 3*x          # operación in-place
```

```

    fx += 4
    return fx

y_inplace = g(x_array)
print("In-place:", y_inplace)

```

For-loop: [np.int64(4), np.int64(2), np.int64(2), np.int64(4), np.int64(8), np.int64(14), np.int64(22), np.int64(32), np.int64(44), np.int64(58)]

Vectorizado: [ 4 2 2 4 8 14 22 32 44 58]

In-place: [ 4 2 2 4 8 14 22 32 44 58]

## Creación de una cuadrícula mediante difusión

En lenguajes no vectorizados, como C, esto se hace con bucles anidados, evitando arreglos temporales, pero el código es más largo.

NumPy permite lograr lo mismo usando broadcasting. En lugar de construir los tres arreglos grandes, se crean tres vectores 1D (para filas, columnas y profundidad) y NumPy automáticamente los combina para formar el arreglo 3D final. Esto reduce la memoria y acelera los cálculos:

Vectorización ingenua: 410 ms

Vectorización con broadcasting: 182 ms

Con broadcasting, el arreglo resultante sigue teniendo la misma forma (200, 200, 200), pero se usan menos recursos de memoria (~128 MB) y menos operaciones temporales.

## Ejemplo en Python usando broadcasting

```

In [13]: import numpy as np

# Crear Los vectores de coordenadas usando ogrid (broadcasting)
i, j, k = np.ogrid[-100:100, -100:100, -100:100]

# Calcular R usando broadcasting
R = np.sqrt(i**2 + j**2 + k**2)

# Verificar la forma del arreglo resultante
print("Forma de R:", R.shape)

# Mostrar algunos valores
print("Valores en R[0,0,:5]:", R[0,0,:5])
print("Valores en R[100,100,100]:", R[100,100,100])

```

Forma de R: (200, 200, 200)

Valores en R[0,0,:5]: [173.20508076 172.62966141 172.05812971 171.49052452 170.92688495]

Valores en R[100,100,100]: 0.0

## Visión artificial

Cuando trabajamos con un conjunto de puntos en 3D representados como un arreglo  $n \times 3$  y una matriz de cámara  $3 \times 3$ , a menudo necesitamos proyectar esos puntos a

coordenadas 2D de píxeles, tal como los vería una cámara. Esto se logra aplicando el producto matricial de cada punto con la matriz de cámara y luego normalizando las coordenadas dividiéndolas por su componente z, que representa la profundidad.

En NumPy, esta operación se realiza de manera eficiente utilizando la función `dot` para el producto matricial, y el broadcasting para dividir cada fila del resultado por su tercer elemento. Gracias a esto, podemos transformar cientos de miles de puntos mucho más rápido que con un bucle tradicional en Python, alcanzando hasta 70 veces la velocidad. Además, el código permanece claro y conciso, sin perder control sobre la memoria utilizada.

Sin embargo, es importante recordar que la vectorización y el broadcasting no son siempre la solución óptima. Cuando se realizan operaciones repetidas sobre grandes bloques de memoria, puede ser más eficiente combinar un bucle externo con un bucle interno vectorizado, para aprovechar mejor la caché del sistema y reducir posibles cuellos de botella.

En conjunto, NumPy nos permite trabajar con grandes conjuntos de datos de manera rápida y eficiente, proyectando puntos 3D a 2D de manera clara, sin sacrificar rendimiento ni claridad en el código.

## Ejemplo práctico

```
In [15]: import numpy as np

# Crear 100,000 puntos 3D aleatorios
points = np.random.random((100000, 3))

# Matriz de cámara 3x3
camera = np.array([[500., 0., 320.],
                   [ 0., 500., 240.],
                   [ 0., 0., 1.]])

# Producto matricial
vecs = camera.dot(points.T).T

# Proyección a coordenadas de píxel 2D usando broadcasting
pixel_coords = vecs / vecs[:, 2, np.newaxis]

# Mostrar los primeros 5 píxeles
print(pixel_coords[:5])
```

```
[[1.17396917e+03  9.75842815e+02  1.00000000e+00]
 [4.03684064e+02  5.61800935e+02  1.00000000e+00]
 [6.64231454e+03  2.03388894e+03  1.00000000e+00]
 [1.44818834e+03  9.82022102e+02  1.00000000e+00]
 [4.15640860e+04  3.36658220e+04  1.00000000e+00]]
```

El uso de arreglos de NumPy demuestra ser fundamental para trabajar con grandes cantidades de datos de manera eficiente, especialmente en aplicaciones de visión artificial. Al proyectar puntos 3D a coordenadas 2D de píxeles, NumPy permite combinar producto matricial y broadcasting para realizar operaciones complejas de manera rápida y sin sacrificar la claridad del código.

La vectorización y el broadcasting facilitan transformar cientos de miles de puntos mucho más rápido que con bucles tradicionales en Python, optimizando tanto el tiempo de ejecución como el uso de memoria. Esto hace posible manejar grandes volúmenes de datos, como los que se encuentran en imágenes, modelos 3D o simulaciones, de manera práctica y eficiente.

Sin embargo, estas técnicas no siempre son la solución definitiva. En operaciones repetidas sobre bloques de memoria muy grandes, puede ser más efectivo combinar un bucle externo con un bucle interno vectorizado, aprovechando mejor la caché del sistema y evitando cuellos de botella.

En resumen, NumPy proporciona herramientas poderosas para el procesamiento de datos multidimensionales, haciendo posible trabajar con conjuntos de datos masivos de forma rápida, eficiente y controlada, manteniendo la legibilidad y flexibilidad del código, lo que es esencial en campos como la visión artificial y la simulación computacional.

[Repositorio segundo parcial](#)